

Лабораторна робота №3

Розробка серверної частини (Back-end) та API

Мета роботи — реалізувати шар доступу до даних, бізнес-логіку та надати зовнішній інтерфейс для взаємодії з системою, використовуючи один із сучасних стандартів (REST, GraphQL або gRPC).

1. Реалізація шару доступу до даних (Data Access Layer)
2. Реалізація бізнес-логіки (Service Layer)
3. Реалізація API (Оберіть один із варіантів)
4. Використання DTO (Data Transfer Objects)

Скріншоти роботи API

Clinic API

1.0.0 OAS 3.0

API для інформаційно-довідкової системи медичного закладу

Servers

http://localhost:5000 - Локальний сервер

Authorize

Appointments

POST /api/appointments Створити запис на прийом (тільки пацієнт)

GET /api/appointments/my Отримати свої записи (пацієнт)

GET /api/appointments/doctor Отримати записи лікаря

PATCH /api/appointments/{id}/status Оновити статус запису

DELETE /api/appointments/{id} Скасувати запис

Auth

POST /api/auth/register Реєстрація нового користувача

POST /api/auth/login Вхід в систему

Doctors

GET /api/doctors Отримати список всіх лікарів

POST /api/doctors Додати лікаря (тільки адмін)

GET /api/doctors/{id} Отримати лікаря за ID

PATCH /api/doctors/{id} Оновити дані лікаря (тільки адмін)

DELETE /api/doctors/{id} Видалити лікаря (тільки адмін)

Symptoms

GET /api/symptoms Отримати всі симптоми

POST /api/symptoms Додати симптом (тільки адмін)

POST /api/symptoms/recommend Отримати рекомендації лікаря за симптомами

Users

GET /api/users/me Отримати свій профіль

PATCH /api/users/me Оновити свій профіль

DELETE /api/users/{id} Видалити користувача (тільки адмін)

1. Приклади коду (фрагменти у звіті):

- Визначення моделі API (DTO / Proto message / GraphQL Type).

AppointmentDto.ts: DTO для запису на прийом

```
export interface AppointmentDTO {
  id: number;
  appointmentDate: Date;
  status: string;
  notes: string | null;
  doctor: {
    id: number;
    firstName: string;
  };
}
```

```

    lastName: string;
    specialization?: {
      name: string;
    };
  };
};
user: {
  id: number;
  firstName: string;
  lastName: string;
};
}

export const toAppointmentDTO = (appointment: any): AppointmentDTO => ({
  id: appointment.id,
  appointmentDate: appointment.appointmentDate,
  status: appointment.status,
  notes: appointment.notes,
  doctor: {
    id: appointment.doctor.id,
    firstName: appointment.doctor.firstName,
    lastName: appointment.doctor.lastName,
    specialization: appointment.doctor.specialization
      ? { name: appointment.doctor.specialization.name }
      : undefined,
  },
  user: {
    id: appointment.user.id,
    firstName: appointment.user.firstName,
    lastName: appointment.user.lastName,
  },
});

```

DoctorDto.ts: DTO для лікаря

```

export interface DoctorDTO {
  id: number;
  firstName: string;
  lastName: string;
  email: string;
  phone: string | null;
  specialization: {
    id: number;
    name: string;
  };
}

export const toDoctorDTO = (doctor: any): DoctorDTO => ({

```

```
id: doctor.id,  
firstName: doctor.firstName,  
lastName: doctor.lastName,  
email: doctor.email,  
phone: doctor.phone,  
specialization: {  
  id: doctor.specialization.id,  
  name: doctor.specialization.name,  
},  
});
```

SymptomDto.ts: DTO для симптомів

```
export interface SymptomDTO {  
  id: number;  
  name: string;  
  description: string | null;  
}  
  
export const toSymptomDTO = (symptom: any): SymptomDTO => ({  
  id: symptom.id,  
  name: symptom.name,  
  description: symptom.description,  
});
```

UserDto.ts: DTO для користувача

```
export interface UserDTO {  
  id: number;  
  firstName: string;  
  lastName: string;  
  email: string;  
  role: string;  
  createdAt: Date;
```

```

}

export interface AuthResponseDTO {
  user: UserDTO;
  token: string;
}

export const toUserDTO = (user: any): UserDTO => ({
  id: user.id,
  firstName: user.firstName,
  lastName: user.lastName,
  email: user.email,
  role: user.role,
  createdAt: user.createdAt,
});

```

B. Реалізація методу (Controller Action / Resolver / gRPC Service method).

auth.ts — реєстрація користувача

```

import { Router, Request, Response } from "express";
import { authService } from "../services/authService";

const router = Router();

router.post("/register", async (req: Request, res: Response) => {
  try {
    const result = await authService.register(req.body);
    res.status(201).json(result);
  } catch (error: unknown) {
    if (error instanceof Error) {
      res.status(400).json({ message: error.message });
    }
  }
});

router.post("/login", async (req: Request, res: Response) => {

```

```

    try {
      const result = await authService.login(req.body);
      res.json(result);
    } catch (error: unknown) {
      if (error instanceof Error) {
        res.status(400).json({ message: error.message });
      }
    }
  });
});

export default router;

```

doctors.ts — отримання списку лікарів

```

import { Router, Request, Response } from "express";
import {
  getAllDoctors,
  getDoctorById,
  createDoctor,
} from "../repositories/doctorRepository";
import { authMiddleware } from "../middleware/authMiddleware";
import { roleMiddleware } from "../middleware/roleMiddleware";
import { toDoctorDTO } from "../dto/doctorDto";
import prisma from "../lib/prisma";

const router = Router();

router.get("/", async (req: Request, res: Response) => {
  try {
    const doctors = await getAllDoctors();
    res.json(doctors.map(toDoctorDTO));
  } catch (error: unknown) {
    if (error instanceof Error) {
      res.status(500).json({ message: error.message });
    }
  }
});

router.get("/:id", async (req: Request, res: Response) => {
  try {
    const doctor = await getDoctorById(Number(req.params.id));
    if (!doctor) {
      res.status(404).json({ message: "Лікаря не знайдено" });
      return;
    }
  }
});

```

```

        res.json(toDoctorDTO(doctor));
    } catch (error: unknown) {
        if (error instanceof Error) {
            res.status(500).json({ message: error.message });
        }
    }
});

router.post(
    "/",
    authMiddleware,
    roleMiddleware("ADMIN"),
    async (req: Request, res: Response) => {
        try {
            const doctor = await createDoctor(req.body);
            res.status(201).json(doctor);
        } catch (error: unknown) {
            if (error instanceof Error) {
                res.status(400).json({ message: error.message });
            }
        }
    }
);

router.patch(
    "/:id",
    authMiddleware,
    roleMiddleware("ADMIN"),
    async (req: Request, res: Response) => {
        try {
            const doctor = await getDoctorById(Number(req.params.id));
            if (!doctor) {
                res.status(404).json({ message: "Лікаря не знайдено" });
                return;
            }
            const updated = await prisma.doctor.update({
                where: { id: Number(req.params.id) },
                data: req.body,
            });
            res.json(updated);
        } catch (error: unknown) {
            if (error instanceof Error) {
                res.status(400).json({ message: error.message });
            }
        }
    }
);

```

```

router.delete(
  "/:id",
  authMiddleware,
  roleMiddleware("ADMIN"),
  async (req: Request, res: Response) => {
    try {
      const doctor = await getDoctorById(Number(req.params.id));
      if (!doctor) {
        res.status(404).json({ message: "Лікаря не знайдено" });
        return;
      }
      await prisma.doctor.delete({ where: { id: Number(req.params.id) } });
      res.status(200).json({ message: "Лікаря видалено" });
    } catch (error: unknown) {
      if (error instanceof Error) {
        res.status(400).json({ message: error.message });
      }
    }
  }
);

export default router;

```

appointments.ts — створення запису на прийом

```

import { Router, Request, Response } from "express";
import { authService } from "../services/authService";

const router = Router();

router.post("/register", async (req: Request, res: Response) => {
  try {
    const result = await authService.register(req.body);
    res.status(201).json(result);
  } catch (error: unknown) {
    if (error instanceof Error) {
      res.status(400).json({ message: error.message });
    }
  }
});

router.post("/login", async (req: Request, res: Response) => {

```



```

    try {
      const result = await authService.login(req.body);
      res.json(result);
    } catch (error: unknown) {
      if (error instanceof Error) {
        res.status(400).json({ message: error.message });
      }
    }
  });
});

export default router;

```

symptoms.ts — отримання рекомендацій за симптомами

```

import { Router, Request, Response } from "express";
import { getAllSymptoms, createSymptom } from
"../repositories/symptomRepository";
import { recommendationService } from "../services/recommendationService";
import { authMiddleware } from "../middleware/authMiddleware";
import { roleMiddleware } from "../middleware/roleMiddleware";
import { toSymptomDTO } from "../dto/symptomDto";

const router = Router();

router.get("/", async (req: Request, res: Response) => {
  try {
    const symptoms = await getAllSymptoms();
    res.json(symptoms.map(toSymptomDTO));
  } catch (error: unknown) {
    if (error instanceof Error) {
      res.status(500).json({ message: error.message });
    }
  }
});

router.post("/recommend", async (req: Request, res: Response) => {
  try {
    const { symptomIds } = req.body;
    const recommendations = await
recommendationService.getRecommendations(symptomIds);
    res.json(recommendations.map(toSymptomDTO));
  } catch (error: unknown) {
    if (error instanceof Error) {
      res.status(400).json({ message: error.message });
    }
  }
});

```

```
    }  
  }  
});  
  
router.post(  
  "/",  
  authMiddleware,  
  roleMiddleware("ADMIN"),  
  async (req: Request, res: Response) => {  
    try {  
      const symptom = await createSymptom(req.body);  
      res.status(201).json(symptom);  
    } catch (error: unknown) {  
      if (error instanceof Error) {  
        res.status(400).json({ message: error.message });  
      }  
    }  
  }  
);  
  
export default router;
```